

Lab Observation Report: Task 3

UPI Payment System Implementation using Object-Oriented Programming & Custom Exceptions

Course / Topic:	Java Programming – OOP, Interfaces & Exception Handling	Task ID:	Observation Task 3
Language:	Java (JDK 8 or higher)	Execution Mode:	Console Application (CLI)

1. Objective

The primary objective of this project is to design and implement a robust, secure, and modular digital wallet payment system in Java. The application demonstrates core Object-Oriented Programming (OOP) concepts including Encapsulation, Abstraction, and Interface realization, alongside robust Custom Exception Handling for transaction validation.

- To implement custom user-defined exceptions (`InsufficientBalanceException`, `InvalidUPIException`, `InvalidAmountException`) for realistic business validation.
- To abstract payment operations using a generic `PaymentService` interface.
- To encapsulate user account and balance management within a secure `Wallet` class.
- To handle runtime input/transaction errors gracefully without terminating the application unexpectedly.

2. Systems Requirement

Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor (CPU)	1.6 GHz Dual-Core Processor	2.0 GHz Quad-Core Intel i5 / AMD Ryzen 5 or higher
RAM	2 GB	8 GB or higher
Storage Space	500 MB free hard disk space	1 GB SSD space
Display / Input	Standard Display (1024x768), Keyboard & Mouse	Standard Full HD Display, Keyboard & Mouse

Software Requirements

Software Tool	Specification
Operating System	Windows 10/11, macOS, or Linux (Ubuntu 20.04+)
Java Development Kit (JDK)	JDK 8 / JDK 11 / JDK 17 / JDK 21 installed with <code>JAVA_HOME</code> set
IDE / Code Editor	VS Code, Eclipse, IntelliJ IDEA, or command-line Terminal
Compiler / Runtime	<code>javac</code> (Java Compiler) and <code>java</code> (Java Virtual Machine)

3. Algorithm & Logic Flow

Step 1: Initialization

Initialize the `Scanner` object to capture standard user console input.

Step 2: Wallet Creation

Prompt the user to enter `userName`, `mobileNumber`, and `upiId`. Instantiate a `Wallet` object with an initial balance of `0.0`.

Step 3: Service Abstraction

Create a `PaymentService` interface reference pointing to an instance of `UPIPayment`, passing the created wallet as a dependency.

Step 4: Wallet Top-Up (Add Money)

Prompt the user to enter an amount to deposit. Call `wallet.addMoney(amount)` inside a try-catch block:

- If `amount <= 0`, throw `InvalidAmountException`.
- Otherwise, increment balance and display the updated total.

Step 5: UPI Payment Processing

Prompt for receiver's UPI ID and transfer amount. Invoke `paymentService.pay(receiverUpiId, amount)` wrapped in a try-catch-finally structure:

- **Validation 1 (UPI Format):** Check if UPI ID is non-empty, contains '@', and has characters before and after '@'. If invalid, throw `InvalidUPIException`.
- **Validation 2 (Amount Check):** If payment amount is `<= 0`, throw `InvalidAmountException`.
- **Validation 3 (Solvency Check):** If amount exceeds `wallet.getBalance()`, throw `InsufficientBalanceException`.
- **Execution:** If all checks pass, deduct the amount from the wallet and print success confirmation.
- **Finally Block:** Always print "Transaction attempt completed." regardless of success or exception.

Step 6: Display Final Summary & Exit

Query available balance via `paymentService.checkBalance()`, display full account details via `wallet.displayWalletDetails()`, and close the input stream.

4. UML Diagrams

4.1 Class Diagram

```
+-----+ |
<<interface>> | | PaymentService |
+-----+ | +
pay(receiverUpiId: String, amount: double): void | | throws InvalidUPIException,
InvalidAmountException, InsufficientBalanceException | | + checkBalance(): void |
+-----+ ^ |
implements +-----+
+ | UPIPayment |
+-----+ | -
wallet: Wallet |
+-----+ | +
UPIPayment(wallet: Wallet) | | - isValidUPI(upiId: String): boolean | | + pay(receiverUpiId:
String, amount: double): void | | + checkBalance(): void |
+-----+ | | uses
/ manages v
+-----+ | Wallet
| +-----+ | -
userName: String | | - mobileNumber: String | | - upiId: String | | - balance: double |
+-----+ | +
Wallet(userName: String, mobileNumber: String, upiId: String, initBal: double) | | +
addMoney(amount: double): void throws InvalidAmountException | | + getBalance(): double | | +
deductBalance(amount: double): void | | + getUpiId(): String | | + getUserName(): String | | +
getMobileNumber(): String | | + displayWalletDetails(): void |
+-----+ Exception
Hierarchy +-----+ | java.lang.Exception | +-----+ ^
+-----+ | | +-----+
+-----+ +-----+ | InvalidAmountException | |
InvalidUPIException | InsufficientBalanceException | +-----+
+-----+ +-----+
```

5. Source Code Overview

```
import java.util.Scanner;

// Create customized exceptions
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

class InvalidUPIException extends Exception {
    InvalidUPIException(String message) {
        super(message);
    }
}

class InvalidAmountException extends Exception {
    InvalidAmountException(String message) {
        super(message);
    }
}

// Interface for payment operations
```

```

interface PaymentService {
    void pay(String receiverUpiId, double amount)
        throws InvalidUPIException, InvalidAmountException, InsufficientBalanceException;
    void checkBalance();
}

// Wallet class encapsulating account balance and user info
class Wallet {
    private String userName;
    private String mobileNumber;
    private String upiId;
    private double balance;

    public Wallet(String userName, String mobileNumber, String upiId, double initialBalance) {
        this.userName = userName;
        this.mobileNumber = mobileNumber;
        this.upiId = upiId;
        this.balance = initialBalance;
    }

    public void addMoney(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException("Invalid amount: Amount must be positive.");
        }
        balance += amount;
        System.out.println("Amount added successfully. New Balance: " + balance);
    }

    public double getBalance() {
        return balance;
    }

    public void deductBalance(double amount) {
        balance -= amount;
    }

    public String getUpiId() {
        return upiId;
    }

    public String getUserName() {
        return userName;
    }

    public String getMobileNumber() {
        return mobileNumber;
    }

    public void displayWalletDetails() {
        System.out.println("\n----- Wallet Details -----");
        System.out.println("User Name      : " + userName);
        System.out.println("Mobile Number : " + mobileNumber);
        System.out.println("UPI ID       : " + upiId);
        System.out.println("Balance      : " + balance);
        System.out.println("-----");
    }
}

// UPI Payment implementation
class UPIPayment implements PaymentService {
    private Wallet wallet;

```

```

public UPIPayment(Wallet wallet) {
    this.wallet = wallet;
}

private boolean isValidUPI(String upiId) {
    if (upiId == null || upiId.isEmpty()) return false;
    int atIndex = upiId.indexOf('@');
    if (atIndex <= 0 || atIndex == upiId.length() - 1) return false;
    return true;
}

@Override
public void pay(String receiverUpiId, double amount)
    throws InvalidUPIException, InvalidAmountException, InsufficientBalanceException {

    if (!isValidUPI(receiverUpiId)) {
        throw new InvalidUPIException("Invalid UPI ID: " + receiverUpiId);
    }

    if (amount <= 0) {
        throw new InvalidAmountException("Invalid amount: Payment amount must be
positive.");
    }

    if (amount > wallet.getBalance()) {
        throw new InsufficientBalanceException("Insufficient balance for this
transaction.");
    }

    wallet.deductBalance(amount);
    System.out.println("Payment of " + amount + " to " + receiverUpiId + " successful!");
}

@Override
public void checkBalance() {
    System.out.println("Available Balance: " + wallet.getBalance());
}
}

public class ObservationTask3 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("----- Create Wallet -----");
        System.out.print("Enter User Name: ");
        String name = sc.next();
        System.out.print("Enter Mobile Number: ");
        String mobile = sc.next();
        System.out.print("Enter UPI ID: ");
        String upiId = sc.next();

        Wallet wallet = new Wallet(name, mobile, upiId, 0.0);
        PaymentService paymentService = new UPIPayment(wallet);

        System.out.print("\nEnter amount to add to wallet: ");
        double addAmount = sc.nextDouble();
        try {
            wallet.addMoney(addAmount);
        } catch (InvalidAmountException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

```

        System.out.print("\nEnter receiver UPI ID for payment: ");
        String receiverUpi = sc.next();
        System.out.print("Enter payment amount: ");
        double payAmount = sc.nextDouble();

        try {
            paymentService.pay(receiverUpi, payAmount);
        } catch (InvalidUPIException e) {
            System.out.println("Transaction Failed - " + e.getMessage());
        } catch (InvalidAmountException e) {
            System.out.println("Transaction Failed - " + e.getMessage());
        } catch (InsufficientBalanceException e) {
            System.out.println("Transaction Failed - " + e.getMessage());
        } finally {
            System.out.println("Transaction attempt completed.");
        }

        paymentService.checkBalance();
        wallet.displayWalletDetails();

        sc.close();
    }
}

```

6. Compilation & Execution Steps

Follow the steps below to compile and execute the program via Command Line / Terminal:

Step 1: Save the File

Save the Java source code in a file named exactly `ObservationTask3.java` (matching the public class name).

Step 2: Open Command Prompt / Terminal

Navigate to the directory where `ObservationTask3.java` is saved:

```
cd path/to/your/java/folder
```

Step 3: Compile the Java Code

Execute the Java compiler command:

```
javac ObservationTask3.java
```

This generates bytecode files: `ObservationTask3.class`, `Wallet.class`, `PaymentService.class`, `UPIPayment.class`, and class files for custom exceptions.

Step 4: Execute the Application

Run the compiled class using Java Virtual Machine (JVM):

```
java ObservationTask3
```

7. Program Output Execution Scenarios

Scenario 1: Successful Payment Transaction

```
----- Create Wallet -----
Enter User Name: Alex
Enter Mobile Number: 9876543210
Enter UPI ID: alex@okbank

Enter amount to add to wallet: 1000
Amount added successfully. New Balance: 1000.0

Enter receiver UPI ID for payment: john@okaxis
Enter payment amount: 450.50
Payment of 450.5 to john@okaxis successful!
Transaction attempt completed.
Available Balance: 549.5

----- Wallet Details -----
User Name      : Alex
Mobile Number  : 9876543210
UPI ID         : alex@okbank
Balance        : 549.5
-----
```

Scenario 2: Failed Transaction — Insufficient Balance Exception

```
----- Create Wallet -----
Enter User Name: Sam
Enter Mobile Number: 9123456789
Enter UPI ID: sam@upi

Enter amount to add to wallet: 200
Amount added successfully. New Balance: 200.0

Enter receiver UPI ID for payment: merchant@store
Enter payment amount: 500
Transaction Failed - Insufficient balance for this transaction.
Transaction attempt completed.
Available Balance: 200.0

----- Wallet Details -----
User Name      : Sam
Mobile Number  : 9123456789
UPI ID         : sam@upi
Balance        : 200.0
-----
```

Scenario 3: Failed Transaction — Invalid UPI Format Exception

```
----- Create Wallet -----
Enter User Name: Rahul
Enter Mobile Number: 9988776655
Enter UPI ID: rahul@sbi
```

```
Enter amount to add to wallet: 500
Amount added successfully. New Balance: 500.0

Enter receiver UPI ID for payment: invalid_upi_format
Enter payment amount: 100
Transaction Failed - Invalid UPI ID: invalid_upi_format
Transaction attempt completed.
Available Balance: 500.0

----- Wallet Details -----
User Name      : Rahul
Mobile Number  : 9988776655
UPI ID        : rahul@sbi
Balance       : 500.0
-----
```